PROJECT CHARTER

1. General Information

Project Name: Villa Booking Management System
Course: Software Project Management (B20E)
Topic: Villa Booking Management System
Team Size: 6 members (1 Project Manager, 1 QA, 1 Tester, 3 Developers)
Project Duration: 6 Weeks (10/12/2025 – 14/01/2026)
Methodology: Scrum (Agile)
Management Tool: Trello
Supervising Lecturer: Ths. Lê Viết Linh

Deliverables:

- A fully functional villa booking application (demo version).
- Complete project management documentation.

2. Project Reason

Through the team's observation and academic analysis, it is recognized that the demand for online villa booking has been increasing significantly as customers expect faster service, reduced manual procedures, and improved travel experiences. However, many existing booking systems still face several limitations.

- **Fragmented Booking Channels:** Customers often have to access multiple platforms, make phone calls, or exchange emails to search and book villa accommodations. This creates a time-consuming process and leads to inconsistent and unreliable information across different channels.
- **Limited Visibility and Transparency:** Potential customers cannot easily compare available villas, check real-time room availability, or access detailed and verified information such as amenities and pricing. This uncertainty reduces customer confidence and may cause villa owners to lose potential bookings.
- **Outdated Management Practices:** Many villa owners and small management units still rely on spreadsheets, manual calendars, and disconnected communication tools. These practices increase the risk of double bookings, pricing errors, and inefficient occupancy management, while also limiting the ability to respond quickly to market demand.

Therefore, the Villa Booking Management System project is proposed to simulate a modern booking platform that improves both customer experience and operational efficiency while providing students with practical project management experience.

3. Purpose & Constraints

Purpose

The primary objective of this project is to develop a robust and user-friendly web application that digitalizes the entire villa booking lifecycle, from searching and viewing room details to booking confirmation. The system allows users to:

- Browse villas and available rooms.
- View detailed room information such as price, capacity, and amenities.
- Check room availability by selected dates.
- Perform online villa bookings.
- Receive booking confirmation notifications.

In addition, the project applies essential project management artifacts, including Requirement, Scope, Schedule, Cost, Risk, Quality, and Configuration management, ensuring that students gain hands-on experience in structured project execution and documentation.

Constraints

- **Time Constraint:** The project must be completed within three sprints according to the academic schedule.
- **Human Resource Constraint:** The team consists of six members with clearly defined roles and responsibilities.
- **Technology Constraint:** The implementation is limited by the team's learning capability and available study time.
- **Scope Constraint:** The product is developed for academic and demonstration purposes only and is not intended for commercial deployment.
- **Resource Constraint:** No real financial budget is used during development.

4. Project Benefits

For Users: The system provides a convenient and easy-to-use online booking experience, allowing users to access information quickly and complete reservations efficiently.

For the Project Team: The project enhances practical understanding of software project management processes, teamwork, communication, and Scrum-based development practices.

For the Course: The project fulfills course learning outcomes (CLOs) and satisfies the final assessment requirements.

5. Stakeholder Analysis

- **End Users:** Customers and staff who use the villa booking system.
- **Development Team:** Responsible for system development, maintenance, and improvements.
- **QA:** Ensures product quality and compliance with requirements.
- **Tester:** Designs and executes test cases to validate system functionality.
- **Lecturer:** Provides academic guidance and evaluates project performance.

6. Identified Risks

- **R1 – Schedule Risk:** Delays in sprint completion may impact overall delivery timeline (High impact).
- **R2 – Technical Risk:** Booking conflicts or data consistency issues may affect system reliability (High impact).
- **R3 – Scope Risk:** Expansion of requirements beyond the original plan may increase workload (Medium impact).
- **R4 – Quality Risk:** High number of functional defects may reduce system stability (Medium impact).
- **R5 – Communication Risk:** Ineffective team communication may slow down progress and reduce productivity (Medium impact).

7. Overall Budget Summary

The total project budget is defined based on the approved Cost Management Plan:

- **Labor Cost:** 63,000,000 VND
- **Tools Cost:** 0 VND (Free tools)
- **Contingency (10%):** 6,300,000 VND
- **Total Project Cost: 69,300,000 VND**

PROJECT MANAGEMENT PLAN

I. SCOPE MANAGEMENT PLAN

1. Objective

The Scope Management Plan defines the framework for identifying, defining, validating, and controlling the project scope throughout the lifecycle of the Villa Booking Management System.

The primary objective of this plan is to ensure that the project delivers only the approved features and agreed deliverables while preventing uncontrolled scope expansion. All scope-related activities must remain aligned with the approved Project Charter, project schedule, and allocated budget.

This plan also establishes a consistent approach for documenting scope, validating deliverables, managing changes, and maintaining stakeholder alignment during project execution.

2. Scope Definition Process

2.1 Requirement Collection

Project requirements are collected from multiple reliable sources to ensure completeness and accuracy, including:

- Course topic requirements (Topic 1).
- UI/UX design mockups created in Figma.
- Q&A sessions and consultations with the supervising lecturer.

The primary techniques applied during requirement collection include:

- Brainstorming sessions among team members.
- Analysis and review of Figma design artifacts.

Output:

All identified requirements are converted into User Stories and maintained in the Product Backlog to support agile planning and tracking.

2.2 Scope Definition

The Product Owner is responsible for defining and managing the project scope through the following activities:

- Identifying Epics and User Stories based on collected requirements.
- Prioritizing backlog items according to business value and learning objectives.
- Allocating User Stories into appropriate sprints based on capacity and priority.

Once approved, the defined scope becomes the baseline for project execution, monitoring, and change control.

3. Work Breakdown Structure (WBS)

The project scope is decomposed into the following major work packages to ensure clear task ownership and effective planning:

3.1 Project Management

- Project Planning
- Sprint Planning
- Daily Stand-up Meetings
- Sprint Reviews and Retrospectives

3.2 Frontend Development

- Home Page
- About Us Page
- Room Categories
- Zone View
- Room Detail View
- Room Card Component
- Booking Flow
- Booking Calendar
- Booking Result

- Checkout
- Payment
- Confirmation
- Services and Events Page
- Tours Page
- Contact Page
- FAQ Page
- Gallery Page

3.3 Backend Development

- Database Design
- Entity Models
- Migrations
- API Controllers
- Zones Controller
- Rooms Controller
- Bookings Controller
- Content Controllers
- Form Controllers
- Business Logic Implementation

3.4 Integration

- Frontend and Backend Integration
- Docker Setup
- Nginx Configuration

3.5 Testing

- Unit Testing

- Integration Testing
- User Acceptance Testing

3.6 Documentation

- Project Charter
- Management Plans
- Technical Documentation
- User Guide

4. Scope Validation

Scope validation ensures that all deliverables meet agreed requirements before formal acceptance.

Activity	Responsible	Timing
Review User Stories	Product Owner	Sprint Planning
Feature Demonstration	Development Team	Sprint Review
Deliverable Acceptance	Lecturer	Final Review

Validation results are recorded and used as inputs for continuous improvement and acceptance decisions.

5. Scope Control

5.1 Scope Change Process

All scope changes follow a controlled change management procedure:

1. Change requests are recorded and tracked in Trello.
2. Impact analysis is conducted by the Scrum Master.
3. Approval or rejection is decided by the Product Owner.
4. The Product Backlog is updated if the change is approved.

5.2 Scope Control Principles

- No new features are added during an active sprint.
- Approved scope changes are implemented only in subsequent sprints.

- All changes must be documented and traceable for audit and tracking purposes.

II. REQUIREMENT MANAGEMENT PLAN

1. Objective

The Requirement Management Plan defines how project requirements are collected, analyzed, documented, prioritized, traced, and controlled throughout the lifecycle of the Villa Booking Management System.

The objective of this plan is to ensure that all requirements are clearly understood, properly documented, traceable, and aligned with project objectives, while minimizing ambiguity, rework, and uncontrolled requirement changes.

This plan supports effective communication among stakeholders and provides a structured foundation for development, testing, and validation activities.

2. Requirement Collection Process

2.1 Requirement Sources

Project requirements are gathered from multiple sources to ensure completeness and accuracy:

Source	Requirement Type
Course Topic (Topic 1)	Functional Requirements
Figma Design	UI/UX Requirements
Lecturer Q&A Sessions	Requirement Clarifications
Team Analysis	Non-functional Requirements

2.2 Collection Techniques

The following techniques are applied during requirement collection:

- **Brainstorming:** Conducted during Sprint Planning sessions to identify features and improvements.
- **Document Analysis:** Reviewing course documents and Figma designs to extract functional and interface requirements.

- **Q&A Sessions:** Clarifying unclear requirements with stakeholders and the supervising lecturer.

3. Requirement Documentation and Traceability

3.1 Requirement Documentation Format

All requirements are documented using the **User Story** format to ensure clarity and user-centered design:

As a [role]

I want [feature]

So that [benefit]

Each User Story includes detailed **Acceptance Criteria** to define measurable completion conditions.

3.2 Requirement Traceability

Requirement traceability is maintained to ensure that each requirement is implemented and tested properly.

Requirement ID	User Story ID	Test Case ID
REQ-001	US-001	TC-001

Traceability ensures consistency across requirement definition, implementation, and verification.

4. Requirement Prioritization

Requirements are prioritized using the **MoSCoW Method** to support effective sprint planning and resource allocation.

Priority	Description
Must	Mandatory features required for system operation
Should	Important features with high business value
Could	Optional features if time permits
Won't	Deferred features not implemented in this phase

5. Requirement Change Management

Requirement changes follow a controlled change process:

1. New requirements are added to the Product Backlog.
2. The Product Owner reviews and prioritizes the request.
3. Approved requirements are scheduled into upcoming sprints based on team capacity.

All changes are documented to maintain transparency and traceability.

III. SCHEDULE MANAGEMENT PLAN

1. Objective

The Schedule Management Plan defines how the project schedule is planned, estimated, monitored, controlled, and reported throughout the lifecycle of the Villa Booking Management System.

The objective of this plan is to ensure that all project activities are completed within the approved six-week timeline (10/12/2025 – 14/01/2026), while maintaining alignment with the project scope, budget constraints, and quality objectives defined in the Project Charter. This plan supports early detection of delays, continuous monitoring of progress, and effective coordination among project stakeholders.

2. Schedule Development Approach

The project follows a Scrum-based scheduling approach with iterative development cycles across a six-week period.

The scheduling process includes:

- Breaking down project scope into User Stories and Tasks during Sprint Planning.
- Estimating workload using Story Points, Planning Poker, and T-shirt sizing techniques.
- Allocating tasks into multiple sprints distributed across the six-week timeline based on priority and team capacity.
- Defining sprint goals, weekly targets, and delivery milestones.

- Establishing a schedule baseline to measure performance and track deviations.

3. Schedule Structure and Timeline
The overall project timeline spans six weeks from 10/12/2025 to 14/01/2026 and is divided into logical execution phases:

Week 1 – Project Initiation and Planning:
Project kickoff, requirement clarification, environment setup, backlog refinement.

Week 2 – Core Development Phase 1:
Implementation of core booking features and database setup.

Week 3 – Core Development Phase 2:
Continued feature development, UI integration, and initial testing.

Week 4 – Integration and Stabilization:
System integration, bug fixing, and performance stabilization.

Week 5 – System Testing and Documentation:
Comprehensive testing, test reporting, and user documentation.

Week 6 – Final Review and Delivery:
Final validation, defect resolution, deployment preparation, and project closure.

Sprint Planning, Daily Stand-up meetings, Sprint Review, and Retrospective activities are conducted continuously throughout the six-week period to maintain agility and transparency.

4. Velocity and Capacity Planning
Team capacity is calculated based on six team members working throughout the project duration.

Velocity is monitored weekly to evaluate productivity and adjust planning.

Estimated capacity:

- **6 members × 6 weeks ≈ 36 person-weeks**
- Capacity is adjusted based on academic schedule and availability.

Velocity metrics are reviewed to support forecasting and workload balancing.

5. Schedule Monitoring and Control
Project progress is monitored continuously using agile tracking mechanisms.

Monitoring activities include:

- Tracking task status on the Trello Kanban board.
- Reviewing weekly progress summaries and burndown charts.
- Conducting daily stand-up meetings to identify blockers.
- Reviewing milestone completion status.

When schedule deviation is detected, corrective actions are applied such as task re-prioritization, resource reallocation, or scope adjustment with Product Owner approval.

6. Delay Management and Recovery

Schedule risks may arise due to underestimated workload, technical complexity, or limited resource availability.

The escalation process includes:

- **Level 1:** Minor delay – Scrum Master supports resolution.
- **Level 2:** Moderate delay – task reassignment or additional support.
- **Level 3:** Critical delay – scope adjustment and stakeholder notification.

Recovery actions include scope reduction for non-critical features, resource reallocation, and controlled overtime if necessary.

7. Roles and Responsibilities

- **Project Manager:** Oversees schedule planning, tracking, and corrective actions.
- **Product Owner:** Reviews and approves deliverables.
- **Scrum Master:** Facilitates daily coordination and removes blockers.
- **Development Team:** Executes assigned tasks according to the schedule.
- **QA / Tester:** Performs testing activities within the planned timeline.

8. Reporting

Schedule status is communicated through:

- Weekly progress reports.
- Sprint review summaries.

- Trello dashboards and burndown charts.

Reports include progress status, risks, issues, and corrective actions and are shared with the Project Manager, Product Owner, and Lecturer.

IV. COST MANAGEMENT PLAN

1. Objective

This Cost Management Plan defines how project costs will be estimated, budgeted, monitored, controlled, and reported to ensure delivery within the approved cost baseline.

2. Cost Estimation Approach

The project applies the **Labor Contract Model** for cost estimation.

Labor cost is calculated using the formula:

Budget = Unit Cost × Effort (person-days)

Reference unit costs (academic assumption):

Role	Unit Cost / Day
Product Owner	600,000 VND
Scrum Master	550,000 VND
Senior Developer	500,000 VND
Developer	450,000 VND
Tester	400,000 VND

3. Cost Baseline and Budget Allocation

The approved cost baseline is distributed as follows:

Cost Element	Allocated Amount (VND)
Labor Cost	63,000,000

Cost Element	Allocated Amount (VND)
Tools	0 (Free)
Contingency (10%)	6,300,000
TOTAL COST BASELINE	**69,300,000**

4. Earned Value Management (EVM)

Earned Value Management (EVM) integrates scope, schedule, and cost to evaluate project performance.

Metric	Formula	Meaning
PV (Planned Value)	Budget × % Planned	Planned cost
EV (Earned Value)	Budget × % Complete	Value of completed work
AC (Actual Cost)	Actual spent	Actual cost
SPI	EV / PV	Schedule performance
CPI	EV / AC	Cost performance

Measurement Milestones

Milestone	% Planned
Mid-Project Review	52%
Final Delivery	100%

5. Cost Control

5.1 Variance Thresholds

Variance	Action
±5%	Monitor

Variance	Action
±10%	Analyze root cause
±15%	Escalate to Product Owner

5.2 Change Control

- All cost-impacting changes must go through a formal Change Request.
- The Product Owner must approve any change exceeding **5% of the total budget**.

6. Roles and Responsibilities

- **Project Manager:** Tracks cost, analyzes variance, and prepares reports.
- **Product Owner:** Approves cost changes and budget decisions.
- **Scrum Master:** Supports progress tracking and forecasting.
- **Team Members:** Provide effort input for accurate cost tracking.

7. Financial Reporting

Financial reports include:

- Planned vs. Actual cost.
- Variance analysis.
- Remaining cost forecast.
- Corrective actions (if applicable).

Reports are issued periodically to the Project Manager, Product Owner, and Lecturer.

V. HUMAN RESOURCE MANAGEMENT PLAN

1. Objective

The Human Resource Management Plan defines how project human resources are organized, assigned, developed, and monitored to ensure effective collaboration and successful project delivery.

This plan ensures that roles and responsibilities are clearly defined, team performance is monitored, and conflicts are resolved efficiently throughout the project lifecycle.

2. Organizational Structure

The project team follows a Scrum-based structure consisting of:

- **Product Owner:** Nguyễn Hoàng Dũng
- **Scrum Master:** Nguyễn Xuân Hiếu
- **Backend Team:** Đặng Gia Khánh, Nguyễn Thành Trung
- **Frontend Team:** Phạm Công Khánh, Bùi Trần Hoàng Huy

The organizational structure supports cross-functional collaboration and fast decision-making.

3. Roles and Responsibilities

- **Product Owner:**
  Defines product vision, prioritizes backlog, approves deliverables, and communicates with the lecturer.
- **Scrum Master:**
  Facilitates Scrum ceremonies, removes blockers, ensures process compliance, and maintains documentation quality.
- **Backend Developers:**
  Design database, implement APIs, handle business logic, and ensure system reliability.
- **Frontend Developers:**
  Design UI/UX, implement user interfaces, ensure responsiveness, and integrate APIs.
- **Tester / QA:**
  Prepares test cases, executes testing, and validates system quality.

A RACI matrix is used to clarify responsibility and accountability across major activities.

4. Skill Development and Competency Management

Team members possess different skill levels in frontend development, backend development, database design, Scrum practices, testing, and UI/UX.

Skill gaps are addressed through:

- Peer learning and knowledge sharing.
- Code reviews and technical discussions.
- Hands-on practice during sprint execution.

5. Resource Availability and Capacity Planning

- Team size: 6 members.
- Average working time: 4–6 hours per day (academic project).
- Team members are available throughout the project with limited public holidays.

Resource availability is monitored to ensure sprint capacity remains stable.

6. Performance Management

Performance is monitored at both individual and team levels:

- **Individual metrics:** Task completion, code quality, collaboration, and participation in Scrum events.
- **Team metrics:** Sprint velocity, bug rate, attendance, and code review coverage.

Continuous feedback is provided during sprint retrospectives.

7. Conflict Resolution

Conflicts are resolved through a structured escalation path:

1. Team discussion.
2. Scrum Master mediation.
3. Product Owner decision.

Resolution techniques include active listening, problem-focused discussion, and time-boxed meetings.

VI. COMMUNICATION MANAGEMENT PLAN

1. Objective

The Communication Management Plan defines how project information is created, exchanged, documented, and monitored to ensure effective coordination among all stakeholders throughout the project lifecycle.

The objective of this plan is to maintain transparency, timely decision-making, consistent information flow, and clear accountability between the project team, Product Owner, and supervising lecturer. Effective communication minimizes misunderstandings, supports risk mitigation, and ensures alignment with the approved project scope, schedule, and quality objectives.

All communication activities must support traceability, accountability, and continuous improvement.

2. Stakeholder Communication Summary

Communication responsibilities are defined based on stakeholder information needs and project governance requirements.

Stakeholder	Information Type	Communication Method	Frequency	Owner
Lecturer	Progress updates, issues	Email, Trello	Weekly	Product Owner
Team Members	Tasks, technical updates	Zalo Group, Daily Stand-up	Daily	Scrum Master
Product Owner	Sprint deliverables	Sprint Review	Per Sprint	Development Team

This structure ensures that the right information is delivered to the right stakeholders at the appropriate time.

3. Communication Channels

The project utilizes multiple communication channels depending on urgency, complexity, and formality:

- **Instant Messaging (Zalo Group):** Used for daily coordination, quick clarifications, and immediate problem resolution.
- **Project Management Tool (Trello):** Used for task assignment, progress tracking, backlog visibility, and workflow transparency.
- **Online Meetings (Google Meet):** Used for sprint events, reviews, retrospectives, and formal discussions.

- **Email:** Used for formal communication, document exchange, approvals, and external coordination.
- **Source Control Platform (GitHub):** Used for code reviews, technical discussions, and traceability of changes.

Response expectations are defined to ensure timely feedback and avoid unnecessary delays.

4. Scrum Events Communication

Communication is structured around Scrum ceremonies to maintain alignment and transparency.

- **Daily Stand-up:** Synchronizes progress, planned activities, and blockers across team members.
- **Sprint Planning:** Defines sprint goals, backlog commitments, workload allocation, and acceptance criteria.
- **Sprint Review:** Demonstrates completed features and gathers stakeholder feedback for validation and improvement.
- **Sprint Retrospective:** Evaluates team collaboration, process effectiveness, and improvement actions.

These events create a continuous feedback loop supporting adaptive planning and quality improvement.

5. Escalation Process

Issues are escalated using a structured escalation path to ensure timely resolution and accountability.

1. Team discussion and initial resolution.
2. Scrum Master facilitation and coordination.
3. Product Owner decision-making for scope, priority, or delivery impacts.
4. Lecturer consultation for critical or unresolved issues.

This escalation model ensures that risks and issues do not negatively impact project objectives.

6. Communication Review and Reporting

Communication effectiveness is reviewed during sprint retrospectives and milestone evaluations.

Reports include progress status, risks, issues, and corrective actions and are shared with the Project Manager, Product Owner, and Lecturer. All key decisions, approvals, and changes are documented to maintain transparency and traceability throughout the project lifecycle.

VII. RISK MANAGEMENT PLAN

1. Objective

The Risk Management Plan defines a structured approach for identifying, analyzing, and managing potential risks that may affect the development and delivery of the Villa Booking Management System.

The objective is to reduce the likelihood and impact of negative events that could compromise project success, while maintaining project stability, quality, and schedule commitments.

2. Risk Detection

Identification Methods

Risks are identified during sprint planning sessions, daily stand-up meetings, sprint reviews, and internal team discussions. Technical reviews and lessons learned from similar academic projects are also considered.

Core Focus

Special attention is given to risks related to booking logic accuracy, data consistency, and online payment integration, as these areas have the highest impact on system reliability and user trust.

3. Risk Classifications

Risks are categorized into the following groups:

- **Technical Risks:** Issues related to system architecture, integration, booking logic correctness, database stability, and payment processing.
- **Schedule Risks:** Risks arising from limited sprint duration, workload estimation errors, and dependency between development tasks.

- **Requirement Change Risks:** Risks caused by scope changes, unclear requirements, or evolving stakeholder expectations.

4. Risk Evaluation

Each identified risk is evaluated based on its probability of occurrence and potential impact on project objectives.

High-impact and high-probability risks are prioritized and addressed earlier in the development lifecycle to reduce uncertainty and avoid late-stage rework. Risk severity is reviewed regularly during sprint planning and sprint review meetings.

5. Risk Reduction

Risk mitigation strategies are defined to minimize exposure and ensure project continuity:

- Core and high-risk features are implemented early to reduce technical uncertainty.
- Incremental testing is performed in every sprint to detect defects and integration issues as early as possible.
- Continuous communication with the Product Owner is maintained to manage expectation changes and requirement adjustments effectively.
- Backup solutions and contingency actions are prepared for critical technical components.

6. Risk Tracking and Monitoring

Risks are reviewed during sprint reviews and regular project checkpoints.

Risk status is continuously updated throughout the project lifecycle to reflect current conditions, such as active, mitigated, or resolved. Emerging risks are recorded and monitored to ensure timely response.

7. Risk Accountability

Each identified risk is assigned a specific owner responsible for monitoring and managing the risk.

The assigned owner is accountable for executing mitigation actions and reporting risk status to the Project Manager or Product Owner when necessary to ensure transparency and timely escalation.

VIII. CONFIGURATION MANAGEMENT PLAN

1. Objective

The Configuration Management Plan defines how project configurations are managed throughout the lifecycle of the Villa Booking System, including source code management, version control, and environment configuration.

The objective of this plan is to ensure consistency, integrity, traceability, and stability of all project artifacts while supporting effective collaboration among team members and preventing uncontrolled changes.

2. Version Control Management

All project source code is stored and maintained in a centralized GitHub repository named **"the-wandering-rose."** All team members are granted read and write access to support collaboration and continuous development.

The project applies a structured branching strategy to separate development activities and maintain system stability:

- **Main branch:** Represents the stable production version used for final delivery.
- **Develop branch:** Used for integration and system testing before release.
- **Feature branches:** Used for implementing individual features such as booking calendar, room detail, and checkout flow.
- **Release branch:** Used to stabilize versions before final deployment.

Branch naming follows a consistent convention to improve clarity and traceability, including feature branches, bug fix branches, release branches, and hotfix branches.

3. Commit Conventions

All commits follow a standardized format to improve traceability and code readability.

Each commit message includes:

- The change type (feature, bug fix, documentation, refactoring, testing, or maintenance).
- The affected scope or module.
- A concise description of the change.

Clear commit conventions support effective code review, auditing, and historical tracking.

4. Code Review Process
All source code changes must be reviewed before being merged into the integration branch.

The review process includes:

- Creating a pull request by the developer.
- Assigning at least one reviewer.
- Verifying coding conventions, correctness, and test results.
- Approving and merging only after successful review.

This process ensures consistent code quality and reduces the risk of defects.

5. Release and Version Management
The project applies semantic versioning using the format **v<major>.<minor>.<patch>.**

- **Major versions** represent breaking changes or major milestones.
- **Minor versions** represent new features and functional improvements.
- **Patch versions** represent bug fixes and minor enhancements.

Releases are aligned with sprint milestones and final delivery requirements to ensure controlled deployment.

6. Environment Configuration Management
Multiple environments are maintained to support different stages of development:

- **Development environment:** Used for feature implementation and local testing.
- **Staging environment:** Used for integration testing and validation.
- **Production environment:** Used for final delivery and demonstration.

Sensitive environment variables such as API URLs and keys are stored locally and excluded from version control to ensure security.

7. Configuration Items Management
Key configuration items include:

- Application source code stored in GitHub.
- Database schema and migration scripts.
- Docker and deployment configuration files.
- API specifications and service definitions.
- Local environment configuration files.

Each configuration item is maintained under version control and assigned clear ownership to ensure accountability and traceability.

IX. CHANGE MANAGEMENT PLAN

1. Objective

The Change Management Plan defines how changes to project scope, requirements, schedule, and technical solutions are formally requested, evaluated, approved, implemented, and documented throughout the lifecycle of the Villa Booking System.

The objective of this plan is to ensure that all changes are controlled, traceable, and aligned with approved project objectives while minimizing disruption to ongoing development activities.

2. Change Identification

Changes may originate from several sources, including:

- Feedback from sprint reviews and system demonstrations.
- New requirements or clarifications from the Product Owner or supervising lecturer.
- Technical improvements proposed by the development team.
- Defects identified during testing activities.
- Risk mitigation or performance optimization needs.

All change requests must be clearly described and documented before evaluation.

3. Change Evaluation and Approval

Each change request is evaluated based on:

- Impact on project scope and functional requirements.

- Impact on schedule and sprint commitments.
- Impact on cost, effort, and resource availability.
- Technical feasibility and risk level.
- Alignment with academic objectives and project priorities.

Approval authority is defined as follows:

- **Scrum Master:** Reviews minor changes with minimal impact.
- **Product Owner:** Approves functional changes and backlog prioritization.
- **Lecturer / Stakeholders:** Approve major changes affecting project baselines.

Unauthorized changes are not permitted.

4. Change Implementation
Approved changes are:

- Added to the Product Backlog.
- Prioritized based on business and learning value.
- Scheduled into future sprints according to team capacity.
- Implemented following standard development, testing, and review processes.

Changes are not introduced during an active sprint unless they are critical.

5. Change Documentation and Traceability
All approved changes are documented with:

- Change description and rationale.
- Approval decision and responsible authority.
- Implementation status and completion confirmation.

Change history is maintained to ensure traceability, transparency, and audit readiness.

6. Change Monitoring and Review
Change effectiveness and impact are reviewed during sprint reviews and retrospectives.

Metrics such as frequency of changes, implementation delays, and defect trends are monitored to improve future planning and maintain system stability.

REQUIREMENT SPECIFICATION

1. Introduction

1.1 Purpose

This document describes the functional and non-functional requirements of the online villa booking system for The Wandering Rose.
It serves as a reference for system design, development, testing, and acceptance.

1.2 Scope

The web-based system allows customers to:

- View villa zones and room information.
- Check room availability by date.
- Make online room reservations.
- View and register for event services and local tours at The Wandering Rose resort.

1.3 Definitions

Term	Definition
Zone	Room area (Wooden House, Rose House, Villa)
Booking	A room reservation
Guest	A customer staying at the villa

2. Functional Requirements

2.1 Module: Room Management

ID	Requirement	Priority
FR-001	The system shall display a list of three zones.	Must
FR-002	The system shall display rooms by selected zone.	Must
FR-003	The system shall display room details including name, price, area, capacity, amenities, and images.	Must
FR-004	The system shall allow filtering rooms by date and number of guests.	Must

2.2 Module: Booking Flow

ID	Requirement	Priority
FR-005	Users shall select check-in and check-out dates using a calendar.	Must
FR-006	Users shall select the number of adults, children, and rooms.	Must
FR-007	The system shall display available rooms.	Must
FR-008	Users shall select rooms and quantity.	Must
FR-009	The system shall calculate and display the total price.	Must
FR-010	Users shall enter personal information (full name, email, phone number).	Must

ID	Requirement	Priority
FR-011	The system shall confirm the booking and generate a booking code.	Must

2.3 Module: Event Services

ID	Requirement	Priority
FR-012	The system shall display a list of four event services.	Should
FR-013	The system shall display service details including description and highlights.	Should
FR-014	Users shall submit service booking requests via a form.	Should

2.4 Module: Tours

ID	Requirement	Priority
FR-015	The system shall display a list of four tours.	Should
FR-016	The system shall display tour details including description and highlights.	Should
FR-017	Users shall submit tour booking requests via a form.	Should

2.5 Module: Information Pages

ID	Requirement	Priority
FR-018	The system shall display the Home page with a hero section.	Must
FR-019	The system shall display the About Us page.	Should
FR-020	The system shall display the Contact page with a contact form.	Should
FR-021	The system shall display the FAQ page.	Could
FR-022	The system shall display the Gallery page.	Could

3. Non-Functional Requirements

3.1 Performance

ID	Requirement	Metric
NFR-001	Page load time shall be less than 3 seconds.	< 3 seconds
NFR-002	API response time shall be less than 500 ms.	< 500 ms

3.2 Usability

ID	Requirement
NFR-003	The system shall support responsive design for mobile, tablet, and desktop.
NFR-004	The user interface shall be in Vietnamese.
NFR-005	The navigation shall be clear and easy to use.

3.3 Reliability

ID	Requirement
NFR-006	System uptime shall be at least 99% after deployment.
NFR-007	Data shall not be lost when the page is refreshed.

3.4 Security

ID	Requirement
NFR-008	The system shall validate all input data.
NFR-009	Sensitive information shall not be stored on the client side.

3.5 Compatibility

ID	Requirement
NFR-010	The system shall support Chrome, Firefox, Safari, and Edge browsers.
NFR-011	The system shall run on Windows, macOS, and Linux operating systems.

4. User Stories

Epic 1: View Room Information

US001 – View zones list

As a customer,
I want to view the list of room zones,
So that I can choose a suitable area.

Acceptance Criteria:

- Display three zones: Wooden House, Rose House, Villa.
- Each zone displays image, name, and short description.
- Users can click a zone to view details.

US002 – View room details

As a customer,
I want to view detailed room information,
So that I can decide whether the room is suitable.

Acceptance Criteria:

- Display name, price, area, and capacity.
- Display amenities list.
- Display image gallery.
- Provide a “Book Now” button.

Epic 2: Booking

US003 – Select booking dates

Acceptance Criteria:

- Calendar allows selection from the current date onward.
- Check-out date must be after check-in date.
- Display total number of nights.

US004 – View available rooms

Acceptance Criteria:

- Filter by date and number of guests.

- Display nightly price.
- Display total price.
- Allow selecting multiple rooms.

US005 – Booking confirmation
Acceptance Criteria:

- Enter personal information.
- Display booking summary.
- Generate booking code after confirmation.

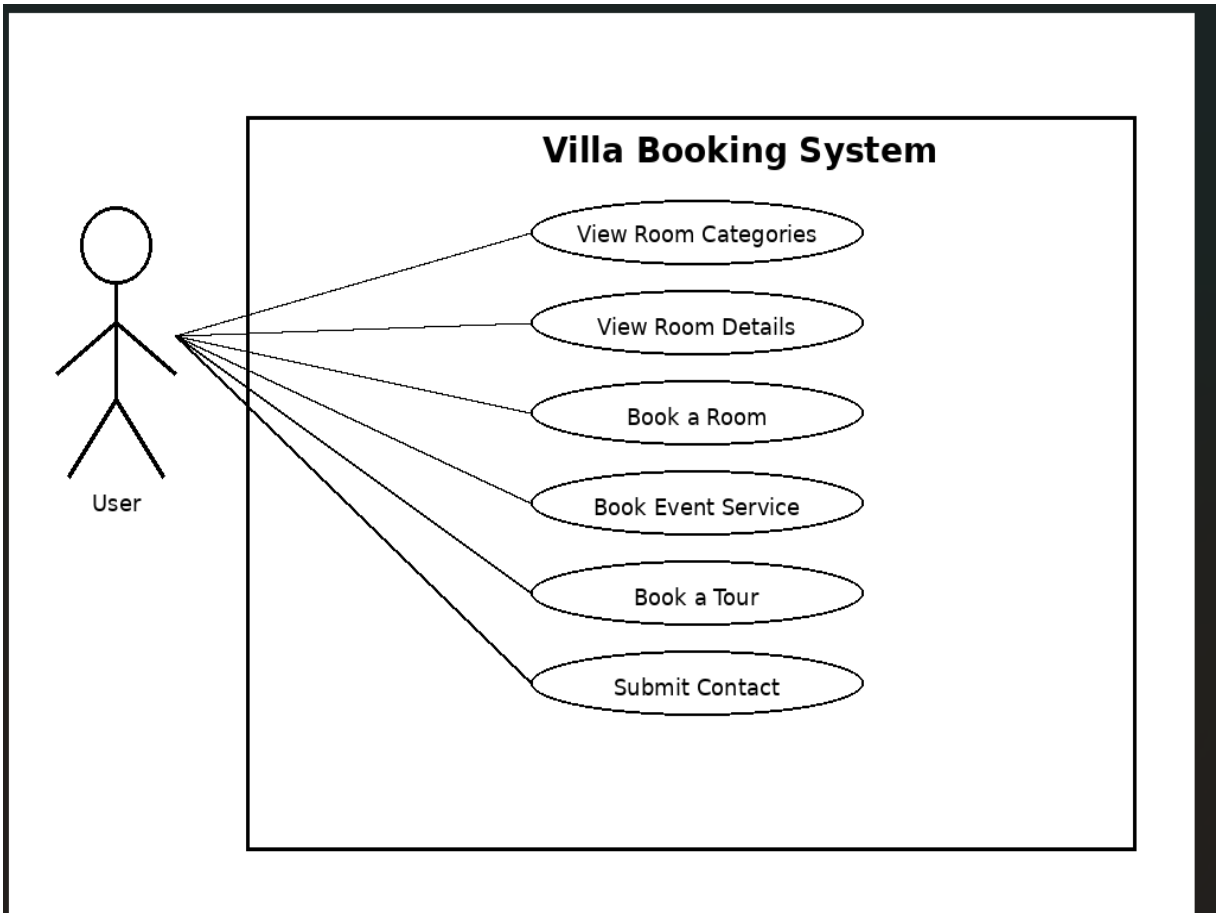
Epic 3: Services and Tours
US006 – View and book event services
Acceptance Criteria:

- Display four services.
- Display service description and highlights.
- Provide inquiry form.

US007 – View and book tours
Acceptance Criteria:

- Display four tours.
- Display tour itinerary and highlights.
- Provide inquiry form.

5. Use Case Diagram



6. Data Requirements

Rooms Table

Field	Type	Required
id	int	Yes
name	string	Yes
maxPeople	int	Yes
area	int	Yes
price	decimal	Yes
zone	string	Yes

Field	Type	Required
imageUrl	string	Yes
description	string	No
features	json	No

Bookings Table

Field	Type	Required
id	int	Yes
bookingCode	string	Yes
checkIn	date	Yes
checkOut	date	Yes
totalPrice	decimal	Yes
status	string	Yes
customerId	int	Yes